

Hooks no Claude Code: da Teoria à Implementação

Documentação profissional dos hooks implementados
e validados nesta sessão — automação determinística
de eventos do Claude Code.

Stop Hook

PreToolUse Hook

Testado end-to-end

Projeto: Claude Code Avançado

Autor: Fernando José da Silva Melo

Data: 08 de setembro de 2026

Gerado por: Claude Sonnet 5 (Claude Code)

01 Visão Geral

Nesta sessão foram implementados dois hooks do Claude Code diretamente no projeto, indo da explicação conceitual até a configuração funcional e testes de ponta a ponta. Hooks são comandos shell disparados automaticamente em eventos do ciclo de vida do Claude Code — como antes/depois de uma tool executar, ou ao final de uma resposta — configurados em `settings.json`.

Por que hooks importam: ao contrário de instruções em prompt ("seja cuidadoso"), hooks são **determinísticos** — o comportamento não depende do modelo decidir seguir uma regra, é imposto por script antes ou depois da ação.

Exemplo	Evento	Objetivo	Status
1 — Notificação sonora	Stop	Tocar um som ao final de cada resposta do Claude	Ativo e testado
2 — Bloqueio de comandos perigosos	PreToolUse (matcher: Bash)	Impedir execução de comandos destrutivos antes de rodarem	Ativo e testado end-to-end

Estrutura de arquivos criada

```
Claude Code Avancado/
├── .claude/
│   ├── settings.json          ← hooks Stop + PreToolUse ativos
│   └── hooks/
│       └── block-dangerous.sh ← script de bloqueio (executável)
└── Hooks/
    ├── Hooks na Pratica/
    │   ├── exemplo-1-som-ao-finalizar/
    │   │   ├── settings.json
    │   │   └── README.md
    │   └── exemplo-2-bloquear-comandos-perigosos/
    │       ├── settings.json
    │       ├── block-dangerous.sh
    │       └── README.md
```

02 Exemplo 1 — Notificação Sonora (Stop)

Hook simples de propósito único: dispara um som toda vez que o Claude termina de processar uma resposta, usando o evento `Stop`.

Configuração

`.claude/settings.json`

```
{
  "hooks": {
    "Stop": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "canberra-gtk-play -i complete 2>/dev/null"
          }
        ]
      }
    ]
  }
}
```

O comando `canberra-gtk-play -i complete` reproduz o som "complete" do tema de sons freedesktop, nativo do ambiente Linux utilizado.

- Testado isoladamente (`exit 0`) e validado no fluxo real — som tocado ao final das respostas subsequentes.

Alternativas por sistema operacional

Ambiente	Comando
macOS	<code>afplay /System/Library/Sounds/Glass.aiff</code>
Linux + PulseAudio	<code>paplay /usr/share/sounds/freedesktop/stereo/complete.oga</code>
Linux genérico	<code>ffplay -nodisp -autoexit -loglevel quiet complete.oga</code>

03 Exemplo 2 — Bloqueio de Comandos Perigosos (PreToolUse)

Hook de segurança que intercepta toda chamada da tool **Bash** antes de ela executar. O script recebe o JSON da chamada via stdin, extrai o comando com **jq** e verifica padrões destrutivos via regex. Ao encontrar um match, retorna **exit 2** — o Claude Code cancela a execução automaticamente.

Script de bloqueio

.claude/hooks/block-dangerous.sh

```
#!/bin/bash
input=$(cat)
command=$(echo "$input" | jq -r '.tool_input.command // ""')

if echo "$command" | grep -qE 'rm[[:space:]]+-rf[[:space:]]+(/[[:space:]]|$)|push[[:space:]]+
  echo "Bloqueado pelo hook PreToolUse: comando potencialmente destrutivo detectado ($command)"
  exit 2
fi

exit 0
```

Registro no settings.json

.claude/settings.json

```
"PreToolUse": [
  {
    "matcher": "Bash",
    "hooks": [
      { "type": "command", "command": "bash .claude/hooks/block-dangerous.sh" }
    ]
  }
]
```

Padrões bloqueados

- **rm -rf /** (e variações com barra/espço final)
- **git push --force** contra **main** / **master**
- **git reset --hard**

Bateria de testes

Comando testado	Resultado esperado	Resultado obtido
rm -rf /	Bloqueado	exit 2 — bloqueado

<code>git push --force origin main</code>	Bloqueado	exit 2 — bloqueado
<code>ls -la</code>	Permitido	exit 0 — permitido
<code>rm -rf node_modules</code>	Permitido	exit 0 — permitido

Validação end-to-end

Além dos testes isolados do script, o hook foi validado disparando um comando real através do próprio Claude Code:

```
$ rm -rf /  
  
PreToolUse:Bash hook error: [bash .claude/hooks/block-dangerous.sh]:  
Bloqueado pelo hook PreToolUse: comando potencialmente destrutivo detectado (rm -rf /)
```

● O comando foi interceptado e nunca chegou a executar — comportamento determinístico confirmado em produção.

Limitações conhecidas: o filtro é baseado em regex sobre a string do comando e pode ser contornado por ofuscação (variáveis, `$(...)`, comandos equivalentes não previstos). Funciona como uma rede de segurança adicional, não como sandboxing completo. Recomenda-se ajustar os padrões ao contexto do projeto (branches protegidas, paths sensíveis, etc.).

04 Conclusão

Os dois exemplos cobrem os dois principais estilos de hook no Claude Code:

- **Notificação (Stop)** — feedback passivo, não interfere no fluxo de execução.
- **Guarda de segurança (PreToolUse)** — controle ativo, capaz de interromper ações antes que aconteçam.

Ambos estão ativos no `settings.json` do projeto e documentados com READMEs individuais em `Hooks/Hooks na Pratica/`, servindo como material de referência reutilizável para o curso.

Possíveis próximos passos

- Expandir o script de bloqueio para cobrir `curl | bash`, `sudo`, `chmod 777`.
- Adicionar um hook `PostToolUse` para formatação automática após edições.
- Explorar o evento `Notification` para alertas externos (Slack, desktop).

Relatório gerado automaticamente por Claude Sonnet 5 dentro do Claude Code — documenta fielmente as ações executadas e validadas nesta sessão de trabalho.